

MacroWatch

README — Architecture, File Structure & System Overview

Version: v1.1-stable | Live URL: <https://macro-watch-phi.vercel.app>

GitHub: <https://github.com/mlibrizzi/MacroWatch>

What is MacroWatch?

MacroWatch is a personal macro intelligence dashboard for monitoring US debt cycle dynamics through Ray Dalio's Big Debt Cycle framework. It aggregates data from 8+ APIs into a unified mobile-first PWA (Progressive Web App) deployed on Vercel and installable on iPhone, Android, and desktop.

The dashboard answers one central question: Where are we in the long-term debt cycle, and what does that mean for capital preservation?

System Architecture

Architecture Diagram

User Device (iPhone/iMac/Android)

↓ HTTPS request to macro-watch-phi.vercel.app

Vercel CDN + Serverless Functions

↓ React/Vite frontend (static bundle)

↓ Serverless API routes (/api/*.js)

↓ External API calls (server-side, keys protected)

External APIs: *Anthropic, FRED, Finnhub, gold-api.com, OilPriceAPI, open.er-api.com, Treasury, TIC*

Component Roles

Component	Role	Details
Anthropic Claude	AI narrative generation	Powers Intel briefing, Fed narrative text, government report summaries. Model: claude-3.5-sonnet
GitHub	Source code repository	Repo: mlibrizzi/MacroWatch . All code changes pushed via GitHub API from iMac Terminal
Vercel	Hosting + serverless functions	Free hobby tier. Auto-deploys on every GitHub push. Serves static React bundle + /api routes
React/Vite	Frontend framework	v18 React, Vite build tool, Recharts for charts, deployed as static bundle with PWA service worker
FRED API	Macro data backbone	Treasury yields, CPI, PCE, GDP, VIX, Fed balance sheet, TGA, oil prices (historical & real-time)
Finnhub	Market data	Real-time Mag7 stock prices, SPX/NDX/DJI via ETF proxy, quarterly earnings vs estimates
gold-api.com	Precious metals	Real-time gold and silver spot prices. Free tier.

OilPriceAPI.com	Oil prices	Near real-time WTI and Brent crude (~30 min refresh). Free tier.
Treasury Fiscal Data	Auction results	Official Treasury auction data: BTC, indirect bidders, dealer takedown, high yield. Fr
Treasury TIC	Foreign holdings	slt_table5.txt — major foreign holders of US Treasuries. Official, free, 45-day lag.
open.er-api.com	FX rates	EUR/USD, USD/JPY, GBP/USD. Free, hourly update.

File Structure

GitHub Repository: mlibrizzi/MacroWatch

Root Directory

File/Folder	Purpose
package.json	Node.js dependencies and build scripts
vite.config.js	Vite build configuration (React plugin)
vercel.json	Vercel deployment configuration (if present)
index.html	HTML entry point for the React app
.env (local only)	Local environment variables — NOT committed to GitHub

/api — Vercel Serverless Functions

File	Purpose	External API Called
claude.js	Anthropic API proxy. Adds system prompt enforcement and strips markdown fences from responses	anthropic.com
fred.js	Fetches all FRED data: yields (2Y/10Y/30Y/TIPS), inflation (CPI/PPI), labor (unemployment/NFP/claims), GDP	fred.stlouisfed.gov
markets.js	Fetches Finnhub data: Mag7 stocks, SPX/NDX/DJIA, EUR/GBP, EUR/USD, JPY/USD	finnhub.io
metals.js	Fetches real-time gold and silver spot prices	gold-api.com
oil.js	Fetches near real-time WTI and Brent crude prices with oil price change	api.oilprice.com
auctions.js	Fetches Treasury auction results: BTC, indirect%, dealer bid, high yield, calculates WEAK/MIXED/STRONG/P	treasury.gov
earnings.js	Fetches quarterly earnings: EPS actual vs estimate, earnings miss %, for AAPL/MSFT/NVDA/GOOGL/AMZN/ME	earnings.com
tic.js	Parses Treasury TIC slt_table5.txt for major foreign investors, calculates returns for 8 countries with MoM change and tr	treasury.gov

/src — React Frontend Source

File	Purpose
main.jsx	React app entry point. Mounts App component to DOM.
App.jsx	Main application file containing ALL UI components: tab navigation, DailyTab, AuctionsTab, WeeklyTab
fetchers.js	Data orchestration layer. Contains: fetchLive() (all API calls), fetchDaily(), fetchWeekly(), fetchMonthly()
api.js	Anthropic Claude helper. callClaude(prompt, system) function. Sets max_tokens:4000, model, handles

/docs — Documentation (GitHub only)

File	Contents
CheatSheet.md	Living document cheat sheet — data sources, limitations, Dalio framework reference
MacroWatch_Development_Phases.pdf	This document — Phase 1/2/3 plans and completed work
MacroWatch_README.pdf	Architecture overview, file structure, API details
MacroWatch_CheatSheet.pdf	Data reference for all tabs with legends and signal interpretation
MacroWatch_ExternalReferences.pdf	Detailed API documentation, costs, conditions of use

Environment Variables

All API keys are stored as Vercel environment variables (Settings > Environment Variables). Never committed to GitHub. Access in serverless functions via `process.env.KEY_NAME`. Access in React frontend via `import.meta.env.KEY_NAME` (VITE_ prefix required).

Variable Name	Used By	Where to Get It
VITE_ANTHROPIC_API_KEY	api/claude.js, src/api.js	console.anthropic.com > API Keys
VITE_FRED_API_KEY	api/fred.js	fred.stlouisfed.org > My Account > API Keys
VITE_FINNHUB_API_KEY	api/markets.js, api/earnings.js	finnhub.io > Dashboard
VITE_GOLD_API_KEY	api/metals.js	gold-api.com > Dashboard
VITE_OILPRICE_API_KEY	api/oil.js	oilpriceapi.com > Dashboard

How to Make Code Changes

All code changes are made via GitHub API from iMac Terminal using a Personal Access Token. Never paste code directly — always use Python scripts via the heredoc pattern.

Standard Workflow

1. Export GitHub token: `export GH_TOKEN='your_token_here'`
2. Fetch current file from GitHub via `urllib.request`
3. Make surgical string replacement (`code.replace(old, new)`)
4. Verify: print confirmation that change was applied
5. Check for smart quotes: `sum(code.count(chr(c)) for c in [8216,8217,8220,8221])`
6. Upload via GitHub API PUT request with base64 encoded content
7. Wait for Vercel auto-deploy (usually 15-20 seconds)
8. Test via curl or browser before declaring success

GitHub Personal Access Token

Generate at [github.com > Settings > Developer settings > Personal access tokens > Tokens \(classic\)](#). Required scopes: repo (full). Token expires — regenerate when expired. Export fresh each Terminal session.

Stable Version Tags

Tag	Date	Description
v1.0-stable	May 5, 2026	First fully verified baseline. All tabs showing real data. Intel working with hardcoded and
v1.1-stable	May 8, 2026	Real-time oil (OilPriceAPI), cache busting, auction improvements, PENDING status.